RP-03 — Test Architecture & Migration Test Plan: Group Lesson Planner

Metadata

Field	Value
Report id	RP-03
Date	2026-08-20
Subject	Test architecture guarding a vanilla-JS to React migration of a single-file static app
Inputs consumed	RP-01 (`docs/research/rp01-app-inventory/rp01-app-inventory.md`); committed `AGENTS.md`; committed `index.html` (1473 lines) in the research worktree; uncommitted local prior art in the main checkout (`package.json`, `playwright.config.ts`, `tsconfig.json`, 7 spec files, 6 page objects, 17 screenplay files, 7 support modules, 1 fixture module, 4 `docs/*.md`, and a modified 1491-line `index.html`)
Verification statement	Every API, config option, flag and version claim in this report was **externally sourced** from current official documentation or package registries on 2026-08-20 and is cited. The prior-art scaffold was **statically read** in full — 38 TypeScript files totalling 1846 lines, plus 4 configuration files, counted with `find` and `wc`. The falsity of prior-art scenario LP-010 was **independently re-derived statically** from the committed `index.html` (mechanism below). The suite was **not executed** — no test run, no browser session, no `npm install`. Anything reasoned rather than read or sourced is labelled **inference**.
Not verified	Actual suite runtime, actual pass rate, and any behaviour of the React target (which does not exist yet)

Line citations use the **committed, deployed** `index.html` (1473 lines), per RP-01's baseline. The main checkout's working copy is 1491 lines; its line numbers do not match and are never used here.

Executive summary

Six decisions carry this report.

1. **Four layers, not seven.** End-to-end (Playwright), storage contract (Playwright, seeded), pure-function unit (Vitest), and a deliberately thin component layer (Vitest + React Testing Library, React side only). Pixel visual regression is **rejected** for this project: RP-01 established that every icon is an OS-rendered Unicode emoji and that buttons compute to `13.3333px Arial` because no `font: inherit` is set, and Playwright names snapshots per browser *and* platform because rendering differs by OS [S9]. A solo developer on macOS with Ubuntu CI would maintain two baselines per assertion for a UI whose entire stylesheet is 216 lines. Load, API-contract and mutation testing are also rejected — 13,610 gzipped bytes over one request, and no API exists.

2. **Playwright is the right E2E tool and the reason is not habit.** The app's whole surface is browser APIs that jsdom cannot honestly fake: `localStorage` read at module init, `window.confirm` / `alert`, `navigator.clipboard` behind a secure-context requirement, `FileReader` plus `Blob` plus `URL.createObjectURL` download, `<input type=file>` upload, `beforeunload`, and `Intl.NumberFormat` currency output that the tests must assert character-for-character.

3. **Playwright component testing became stable in 1.62 — and is still the wrong choice here.** As of 2026-08-20 the component-testing guide states it replaces the experimental `@playwright/experimental-ct-*` packages, that `fixtures.mount()` is a documented built-in "Added in: v1.62", and that "There is no experimental package to depend on and no separate config dialect" [S3][S4][S5]. The reason to decline it is cost, not maturity: `mount()` requires a **story gallery page you build and serve**, exposing `window.mount(params)` and `window.unmount()` [S4]. That is a second application to maintain, for roughly six components, by one person in personal time. Vitest + RTL needs no extra runtime once Vite exists in the React build.

4. **The single most durable assertion anchor is `localStorage`, not the DOM.** RP-01 §4 pins three keys and one exact persisted shape. Those survive any rewrite by definition — they are the migration's real contract. Every journey test should assert a storage outcome in addition to a visible one.

5. **The existing scaffold cannot run against the deployed application at all.** Its page objects locate group cards by `[data-group-name=…]` and month rows by `[data-month-key=…]`, and read counts through `getByTestId`. RP-01 runtime-verified that the deployed file contains **zero** `data-testid` and zero `data-*` attributes; the diff confirms those attributes exist only in the uncommitted working copy. Therefore the prior art's claim that `npm run test:e2e` passed 22/22 cannot describe the artifact users load. This is the headline finding of §9.

6. **Every test needs an invariant-or-defect tag.** RP-01 quick win 5 changes `(1 lessons)` to `(1 lesson)`; the existing spec asserts the buggy string. RP-01 defects D3, D5 and D14 are behaviours React must *not* reproduce. Without a tag per test, a dual-project run is uninterpretable: the suite either blocks the fixes or silently fails to notice they landed. The exit criterion in §8 is expressed as a diff of two machine-readable reports, not a coverage percentage.

I also verified prior-art scenario LP-010 as false myself, statically, and §3 explains what that implies for trusting the rest of that document.

1. Test architecture

Layer	Tool	Scope	What it catches	What it costs	Include
End-to-end	Playwright + TypeScript	Whole app in a real browser over loopback HTTP	Journey regressions, dialog text, clipboard, CSV download and upload, `Intl` output, unload prompt	Slowest layer; needs a served build and a browser install in CI	**Y** — the only layer where the app's actual dependencies are real
Storage contract	Playwright, context seeded before load	The three keys and the exact persisted shape, both directions	Migration data loss, malformed-input failures, shape drift	Fixture corpus must be curated by hand	**Y** — the highest-value layer for this specific migration
Pure function unit	Vitest, node environment	Extracted CSV parse and serialize, month-key normalisation, override normalisation, template token substitution, weekday arithmetic, currency formatting	Parser and money arithmetic regressions in milliseconds	Nothing is importable today; requires extraction first	**Y**, from Phase 1 onward
Component	Vitest + React Testing Library, jsdom	React only: calendar grid, group-info form, month row	Prop and state wiring, conditional rendering, list keys	New dependency set; jsdom is not a browser	**Y but capped** — three components, not a component suite
Visual regression, pixel	Playwright `toHaveScreenshot`	Rendered pixels	CSS regressions	Separate baseline per browser and OS [S9]; emoji are OS-rendered; buttons resolve to 13.33px Arial	**N** — see below
Structural snapshot	Playwright `toMatchAriaSnapshot`	Accessibility-tree shape of each modal	Accidental structural change during the rewrite	Churns on every intentional markup change	**Y, capped at five** — as a migration diff artifact, not a gate

Layer	Tool	Scope	What it catches	What it costs	Include
Accessibility assertions	`@axe-core/playwright` [S10]	Per-screen WCAG-detectable violations	Regression after the a11y fix work lands	Fails on day one given RP-01 §6	**Y as a ratchet**, N as a pass/fail gate
Cross-browser matrix	Playwright projects	chromium, webkit, firefox	Engine-specific breakage	Triples runtime and flake surface	**Chromium default plus one WebKit smoke lane**; full matrix N
Load and performance	any	page weight, request count	nothing that is not already known	measurement harness	**N** — one request, 13,610 bytes gzipped, no backend
API and contract tests	any	HTTP interfaces	nothing	—	**N** — no API exists
Mutation testing	Stryker	test-suite quality	weak assertions	Multiplies E2E runtime by the mutant count	**N** — disproportionate for a solo maintainer

Why pixel visual regression is rejected

Playwright appends both the browser and the platform to snapshot filenames precisely because "Screenshots differ between browsers and platforms due to different rendering, fonts and more, so you will need different snapshots for them", and advises running "tests in the same environment where the baseline screenshots were generated" [S9]. Three facts from RP-01 §5 make that especially bad here: all six icons are inline Unicode emoji rendered by the host OS, the declared font stack never reaches buttons so they compute to `13.3333px Arial`, and the weekend indicator has a measured contrast ratio of exactly 1.00:1 — that is, the most visually broken thing in the app is *invisible to a pixel diff by construction*. A baseline generated on macOS would fail on `ubuntu-latest` on emoji glyphs alone, so the developer would either maintain two baselines per assertion or run visual tests only in CI and never locally. For a 216-line stylesheet with no design system, the maintenance cost exceeds the defect yield. **Note the disagreement honestly:** Vitest 4 shipped a `toMatchScreenshot` assertion as a headline feature [S25], so the ecosystem is investing in this layer. It is worth reconsidering if and when the React app grows a real component library — not for the migration itself.

The replacement is `toMatchAriaSnapshot`, introduced in 1.49 [S5]: it captures the accessibility tree as YAML, which is text, diffs readably in a pull request, is platform-independent, and expresses exactly the thing a rewrite should preserve — roles, names and structure. The prior art already uses it (`e2e/ui/support/aria-snapshot.ts`), and the developer's own QA document correctly judged that its current snapshots are low-signal when placed on the primary assertion path. Keep them, move them off the primary path, cap the count.

Why the accessibility layer is a ratchet, not a gate

RP-01 §6 records zero `role` attributes, zero `<dialog>` elements, one `aria-*` attribute in the whole document, no landmarks, contrast failures on every accent colour, and non-focusable `div`s for the two primary controls. An axe scan wired as a pass/fail gate would be red on the first run, and a permanently red gate gets deleted. The workable form is a violation **budget file** committed alongside the tests: the test asserts that the violation set is a subset of the recorded baseline, and the baseline only ever shrinks. That converts an unwinnable gate into a monotonic improvement mechanism a solo developer will actually keep.

2. Tooling decision table with recommendation

Decision	Recommended	Alternatives considered	Why it wins here	Version checked 2026-08-20	Version sensitive
E2E runner	`@playwright/test` + TypeScript	Cypress, WebdriverIO, Puppeteer	Real dialogs, downloads, uploads, clipboard, storage seeding, multi-project config, and the maintainer's existing toolchain	1.62.1, engines `node >= 20` [S6]	Yes
Unit runner	Vitest	Jest, node:test	Shares the Vite config the React build will already have; near-zero setup cost	4.1.11, engines node 20/22/24+ [S23]	Yes
Component rendering	React Testing Library on Vitest jsdom	Playwright component testing 1.62, Vitest Browser Mode	No extra runtime to build or serve; see the argument below	`@testing-library/react` 16.3.2, peer React 18 or 19 [S22]	Yes
Visual regression	none (aria snapshots instead)	Playwright `toHaveScreenshot`, Vitest `toMatchScreenshot`	OS-rendered emoji plus per-platform baselines; see §1	aria snapshots since 1.49 [S5]	No
Accessibility engine	`@axe-core/playwright`	manual audit only, pa11y	Playwright's own documented recommendation [S10]	version `TBD` — resolve by reading `https://registry.npmjs.org/@axe-core/playwright/latest`	Yes

Decision	Recommended	Alternatives considered	Why it wins here	Version checked 2026-08-20	Version sensitive
Static server for tests	`http-server` via Playwright `webServer`	`serve`, `vite preview`, `python -m http.server`	Already in the uncommitted `package.json`; `webServer` accepts an array so legacy and React can run side by side [S14][S15]	`http-server ^14.1.1` in local prior art	No
CI host	GitHub Actions	none needed	Free for public repositories on standard runners [S29]; the repo is already public	actions at v7, see §6	Yes
Time control	`page.clock.setFixedTime`	`addInitScript` Date monkey-patch, sinon	First-party, documented, and keeps timers running [S7][S8]	Clock API added in 1.45 [S8]	Yes

Playwright as the E2E choice, validated

The question is whether a static GitHub Pages app with no backend needs a full browser driver. It does, and the reason is the app's dependency list rather than its size. RP-01 §5 enumerates the browser APIs in use: `localStorage`, `Intl.NumberFormat`, `navigator.clipboard`, `FileReader`, `Blob`, `URL.createObjectURL`, `Date`, `confirm`, `alert`. Six of those are the app's primary user-visible outputs, not incidental plumbing:

- The **payment message** is asserted character-for-character, and its money strings come from `Intl.NumberFormat('en-US', {style:'currency'})` which renders `PLN 1,234.50` — a code prefix and a space, not a symbol (RP-01 §7). Asserting that in jsdom asserts Node's ICU build, not the user's browser.
- The **only backup path** is a `Blob` download and a file-input upload. Playwright has first-class `download` events and `setInputFiles`.
- Three destructive flows are guarded by `window.confirm`, and five error paths surface through `alert`. Playwright's `dialog` event handles both; the prior art already does this correctly.
- The clipboard write requires a secure context [RP-01 S8], which is why tests must run over `http://127.0.0.1` rather than `file://`.
- The app reads `localStorage` synchronously inside `App.init()` at `index.html:411`, before anything renders. Seeding must therefore happen **before navigation**, which is exactly what Playwright's `storageState` context option does [S11].

Alternatives only win at other layers, and I name them there rather than pretending they compete for this one: Vitest for pure functions, RTL for component wiring. Neither is an E2E driver for a real browser with real downloads.

Component and unit layer: Vitest + RTL versus Playwright component testing

Playwright's component testing is no longer experimental. Verified on 2026-08-20 by three independent surfaces: the guide's own note box states it "replaces the **experimental** `@playwright/experimental-ct-react` and `@playwright/experimental-ct-vue` packages" and that "There is no **experimental** package to depend on and no separate config dialect" [S3]; the API page carries "Added in: v1.62" for `fixtures.mount()` [S4]; and the 1.62 release notes headline a new component-testing model built on stories and galleries [S5]. The registry confirms 1.62.1 is current [S6]. The first automated read of that guide returned a confident summary that did not surface the note box; the claims above rest on a second, verbatim re-read.

Recommendation: Vitest for pure functions, Vitest + React Testing Library for the three components, and no Playwright component testing. The deciding cost is stated by Playwright itself: `mount(storyId, props)` "mounts a component story" and requires "A **gallery page** … served at the configured baseURL, exposing `window.mount(params)` and `window.unmount()` functions" [S4]. That gallery is a second application — routes, a story registry, a build target, and a serve script — maintained by one person in personal time to test roughly six components extracted from a 1473-line file. Vitest + RTL requires a config block and a `jsdom` dependency, reuses the Vite pipeline the React migration introduces anyway, and runs in seconds without a browser download.

Three qualifications, because the argument is not one-sided:

- **Where jsdom genuinely lies, do not use it.** Anything touching `Intl` currency output, real `localStorage`, clipboard, downloads, or layout stays in the E2E and storage-contract layers, which already run in a real browser. That partition removes jsdom's fidelity gap from the risk register rather than tolerating it.
- **Testing Library's query priority is the same discipline as Playwright's**, which keeps one locator policy across both layers: `getByRole` first, then label and text, and `getByTestId` explicitly described as the "final resort" for "cases where you can't match by role or text or it doesn't make sense" [S21]. §3 explains why this app inverts that default in the short term.
- **Vitest Browser Mode is the middle option and is now stable.** The Vitest 4 announcement states "With this release we are removing the `experimental` tag from Browser Mode" [S25], and the guide lists Playwright as the recommended provider [S24]. It is the correct escalation *if* jsdom fidelity starts costing real debugging time — real browser, Vitest ergonomics, no story gallery. Do not adopt it pre-emptively; there is no evidence yet that jsdom is insufficient for a calendar grid and two forms.

3. Locator and assertion policy

Strategy	Survives a DOM rewrite	Usable on the deployed app today	When to use
`localStorage` key and shape assertions	**Y** — by definition, it is the migration contract	Y	Every journey test, alongside the visible assertion. The strongest anchor in this app

Strategy	Survives a DOM rewrite	Usable on the deployed app today	When to use
`getByTestId` on an agreed contract	**Y**, if the React components emit the same ids	**N** — deployed file has zero `data-testid`	The five repeated collections and their interactive leaves, where roles are ambiguous or absent
`getByRole` plus accessible name	**Y**	**Partially** — see below	Toolbar buttons, modal buttons, `select`, `input type=number`, once scoped to an open container
Visible domain text, exact strings	**Y**, if the vocabulary is preserved	Y	Empty state, counts, totals, dialog and alert text. RP-01 §7 is the authoritative string list
`getByLabel`	**Y**	**N** for four controls — three `<label>` s have neither `for` nor a wrapped control, and `#yearInput` has no label at all	Only after the label associations are fixed
`page.clock` fixed time as an assertion precondition	**Y**	Y	Any test touching today highlighting, the default month, the current-month row, or the export filename
URL, path, query, hash	N/A today, and a *new* anchor after migration, not a preserved one	**N** — RP-01 §5 records zero use of hash, query, `history` or `pushState`	Do not build migration-guard tests on routing. If React adds routes, test them as new behaviour
`#id` CSS selectors	**N**	Y	Never in a migration-guard test. Acceptable only in a throwaway legacy-only probe
Class selectors such as `.day.selected`	**N**	Y	Never. Styling classes are the first thing a rewrite discards
Structural CSS or XPath, `nth-child`, sibling chains	**N**	Y	Never
`page.evaluate(() => App…)`	**N** — RP-01 §5 records that a top-level `const` in a classic script does not survive bundling	Y	Never in a retained test

Strategy	Survives a DOM rewrite	Usable on the deployed app today	When to use
`toMatchAriaSnapshot`	Conditional — survives only if the accessibility tree is deliberately preserved	Y	Capped at five, one per view, as a before-and-after migration diff

Playwright's own guidance is unambiguous and it is the ground for the rows above: the recommended order is `getByRole`, `getByText`, `getByLabel`, `getByPlaceholder`, `getByAltText`, `getByTitle`, `getByTestId`; "Long CSS or XPath chains … are an example of a **bad practice** that leads to unstable tests"; and "CSS and XPath are not recommended as the DOM can often change leading to non resilient tests" [S1]. The best-practices page adds the governing principle: tests should "verify that the application code works for the end users, and avoid relying on implementation details" [S2].

The tension: role locators are partly unavailable until accessibility is fixed

RP-01 §6 and §7 document three concrete blockers, all runtime-verified there:

1. **Group cards and all 31 day cells plus 7 weekday headers are non-focusable `div`s with no `role`.** There is no role to query. These are also the two most-exercised controls in the app.
2. **Closed modals stay in the accessibility tree** — computed `display: flex; visibility: visible` — so on a loaded page the tree simultaneously exposes three buttons named `Cancel` and two named `Save`. `getByRole('button', {name: 'Cancel'})` is ambiguous and will throw on strict-mode violation.
3. **Four form controls have no accessible name**, so `getByLabel` is unavailable for the group name, price, currency and year inputs.

The sequencing that resolves this without waiting on the a11y work:

Step A, immediately usable, no source change. Scope every role query to a container. `getByRole` ambiguity in this app is entirely caused by three overlays coexisting; a query rooted at the open overlay is unambiguous today, before any `hidden` fix. This makes the toolbar, all three modals' buttons, the month `select`, the year `spinbutton` and the two textareas addressable by role right now. The container itself still needs a non-role anchor — that is what Step B provides.

Step B, one commit, then freeze: introduce the test-id contract. Strategy A (test-first) requires anchors in the legacy app. The developer has **already written** most of them in the uncommitted working copy: `data-group-name`, `data-group-index`, `data-month-key`, `data-weekday`, `data-date`, `data-day`, and eight distinct `data-testid` values (`group-card-name`, `group-card-lesson-count`, `month-name`, `month-lesson-count`, `month-total`, `month-price-input`, `price-per-lesson`, `copy-payment-message`) — counted by grep over the working copy, not taken from the brief, which reports seven. Adopt that set rather than inventing a parallel one, and add the four container ids the scoping strategy needs — one per overlay and one for the calendar. Then treat the list as a frozen contract: the React components must emit the same `data-testid` values on the same semantic elements, which is what makes one spec set run against both implementations.

Two rules keep this from degrading into test-id-everywhere. First, a test id is permitted only where a role is absent or ambiguous — that is, the repeated collections and their leaves. Second, once the a11y work lands (roles and `tabindex` on cards and day cells, `hidden` on closed overlays, `for` on the three labels), the *chrome* locators migrate to roles and the test ids remain only on the collections. Testing Library's own framing applies: test ids are for "cases where you can't match by role or text" [S21] — this app currently has an unusually large number of such cases, and that number should shrink.

Step C: prefer accessible-name-driven roles in the React app from the start. The rewrite is the cheap moment to add roles, names and a focus trap. RP-01 §5 notes JSX escaping also removes the stored-XSS class (D1) for free. A React app built role-first needs far fewer test ids than the legacy app does, and the ones it keeps are the collection anchors.

Invariant versus defect: the tag every test must carry

A dual-project run is only interpretable if each test declares what it expects the two implementations to do. Three tags:

Tag	Meaning	Legacy expectation	React expectation
`INV`	Behaviour that must be byte-identical across the rewrite	pass	pass
`DEF`	A defect React must not reproduce	fail, and the failure is recorded as expected	pass
`CHAR`	Characterization of current behaviour that is under product review	pass	pass until the product decision changes the test

Implement the tag with Playwright's grep tags in the title (for example a trailing `@INV`) plus a single exported table mapping each `DEF` test to the RP-01 defect id, so `--grep @DEF` runs exactly the migration-improvement set. Three worked cases:

- `(1 lessons)` and `1 days selected` (RP-01 D19) are `DEF`. The existing spec asserts the buggy strings at `e2e/features/schedule-editing.spec.ts:107` and `:128`, which means RP-01 quick win 5 will turn that spec red. Retag and rewrite to the corrected singular, or assert count-only with a regex, before the fix lands.
- The cross-month bulk price bleed (D3) is `CHAR`, not `DEF`, because RP-01 open question 3 records that the developer does not yet know the intended behaviour. Pin current behaviour so a rewrite cannot change it silently, and re-tag once the product decision is made. Do **not** encode a guess as an invariant.
- Corrupt JSON bricking the page (D14), an unopenable group from a non-3-letter currency (D5) and `5-08-10` month keys (D4) are all `DEF`. These are the tests that give the migration its actual value proposition.

Assertion style

Use web-first assertions throughout — `await expect(locator).toHaveText(…)` rather than reading a value and comparing — because "By using web first assertions Playwright will wait until the expected condition is met" [S2]. This matters concretely here: RP-01 §6 records `setTimeout` delays of 0, 100 and 1000 ms and CSS transitions of 0.2 to 0.3 s on overlays that never change `display`. A polling assertion absorbs all of them. A bare `isVisible()` or a fixed `waitForTimeout` does not, and RP-01's anti-anchor note is decisive: closed overlays compute `display: flex; visibility: visible`, so `toBeVisible()` on a closed overlay is **not** a reliable closed-state assertion in the deployed app. Assert closed state on the `.show` class today, or on the `hidden` attribute once the fix lands, and record which of the two the test relies on.

4. Storage contract test design, including fixture strategy

This is the layer the whole migration rests on. The end user has already lost her data once, and the app's only durable artifact is three `localStorage` keys with no version field, no validation and no error handling (RP-01 §4).

The contract under test

Key	Value encoding	Written by	Read by	Failure mode today
`groupLessonPlannerData`	`JSON.stringify(Array<Group>)`	`save()` at `index.html:1199–1202`	`load()` at `index.html:1188–1198`	unguarded `JSON.parse`; corrupt value aborts `init()` and leaves a dead page (D14)
`groupLessonPlannerSettings`	`JSON.stringify({defaultCurrency})`	same `save()`	same `load()`	unguarded `JSON.parse`; also overwritten by the first CSV row
`paymentTemplate`	**raw string, not JSON**	`setTemplate` at `index.html:1210–1212`	`getTemplate` at `index.html:1207–1209`	empty string is falsy so it silently falls back to the default; survives `clear()` (D15)

Two structural facts a typed model must decide about, both from RP-01 §3: `dates` is **denormalised** — every date exists in `group.dates` and again in `monthlyOverrides[key].dates`, synchronised only in `saveDateChanges` and CSV import — and group identity is the **array index**, while CSV import re-keys by `name`. A migration that silently changes either will corrupt data that reads back without error. Contract tests must assert both arrays, not just the one the UI happens to show.

Test shape: one spec set, two projects

The mechanism that makes "the React version still reads the legacy data" a checkable statement is Playwright's project matrix. `webServer` accepts an array of servers [S14][S15], and per-project `use` accepts any test option including `baseURL` [S16]. Two projects, two ports, one spec directory.

```
import { defineConfig } from '@playwright/test';
```

```
const LEGACY = 'http://127.0.0.1:4173';
const REACT = 'http://127.0.0.1:4174';
const withReact = !!process.env.PW_REACT;

export default defineConfig({
  testDir: 'e2e',
  globalTimeout: 15 * 60 * 1000,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,
  reporter: [['list'], ['html'], ['json', { outputFile: 'test-results/report.json' }]],
  use: {
    timezoneId: 'UTC',
    locale: 'en-US',
    trace: 'retain-on-failure',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'legacy', use: { baseURL: LEGACY } },
    ...(withReact ? [{ name: 'react', use: { baseURL: REACT } }] : []),
  ],
  webServer: [
    { command: 'npm run serve:legacy', url: LEGACY, reuseExistingServer: !process.env.CI,
timeout: 120000 },
    ...(withReact ? [{ command: 'npm run serve:react', url: REACT, reuseExistingServer:
!process.env.CI, timeout: 120000 }] : []),
  ],
});
```

The React project is gated behind an environment flag so Phase 0 runs a single project against the legacy file, and the second project switches on the day a React build exists. `locale: 'en-US'` is added deliberately: RP-01 §7 records that all formatting is pinned to `en-US` in source, and pinning the context locale too [S11] removes the runner machine's locale as a variable in currency assertions.

The seeding fixture

Seed by overriding the built-in `storageState` **option** rather than by constructing a context. Playwright documents both that `storageState` accepts an object with `origins[].localStorage[{name, value}]` [S11] and that fixtures can be overridden, with an explicit example of replacing `storageState` with a fixture [S12][S13]. The origin must be derived per project, because the two projects have different ports.

```
import { test as base } from '@playwright/test';

export type PlannerState = {
  groups: unknown[];
  settings?: { defaultCurrency: string };
  template?: string;
  rawData?: string;
};

export const FIXED_NOW = new Date('2026-03-15T09:00:00Z');

export const test = base.extend<{ plannerState: PlannerState }>({
```

```
  plannerState: [{ groups: [] }, { option: true }],
  storageState: async ({ plannerState }, use, testInfo) => {
    const origin = new URL(String(testInfo.project.use.baseURL)).origin;
    const items = [
      {
        name: 'groupLessonPlannerData',
        value: plannerState.rawData ?? JSON.stringify(plannerState.groups),
      },
    ];
    if (plannerState.settings) {
      items.push({
        name: 'groupLessonPlannerSettings',
        value: JSON.stringify(plannerState.settings),
      });
    }
    if (plannerState.template !== undefined) {
      items.push({ name: 'paymentTemplate', value: plannerState.template });
    }
    await use({ cookies: [], origins: [{ origin, localStorage: items }] });
  },
});
```

The `rawData` escape hatch exists so malformed-value fixtures can plant a string that is not valid JSON, which is the whole point of the D14 test. Note that the built-in `context` fixture is left untouched — §9 explains why the prior art's replacement of it is a defect.

Time control, and the two races RP-01 warned about

RP-01 §6 lists `new Date()` at module init (`index.html:369–370`), in today highlighting, in `jumpToToday`, in the schedule editor's default month, in the current-month cutoff inside `updateDefaultPrice`, and in the export filename. Module init runs during page load, so the clock must be fixed **before** navigation, which the Clock guide states directly: "For best results, install the clock before navigating the page" [S7]. Do not do this inside an automatic fixture — the ordering between an auto fixture and the lazily-created `page` fixture is not documented, and an ordering bug here fails silently. Make it explicit at the one place every test already calls:

```
import type { Page } from '@playwright/test';
import { FIXED_NOW } from './fixtures';

export const openApp = async (page: Page, at: Date = FIXED_NOW) => {
  await page.clock.setFixedTime(at);
  await page.goto('/');
};
```

Use `setFixedTime`, not `install`. This is the non-obvious call. `setFixedTime` "Makes `Date.now` and `new Date()` return fixed fake time at all times, keeps all the timers running" [S8]. That is exactly the combination this app needs: the module-init date becomes deterministic, so the default calendar month, the today highlight, the always-rendered current-month row and the export filename all stop drifting — while the 1000 ms `Copied!` window at `index.html:909`, the three 100 ms focus delays and the 0.2 to 0.3 s overlay transitions continue to elapse normally. `install` combined with `pauseAt` would freeze those

timers, so `Copied!` would never revert and the review modal would never close: the test would hang rather than stabilise. `clock` is available on both `Page` and `BrowserContext`, and was added in 1.45 [S8] [S20].

Two further race rules. Never assert on the transient `Copied!` label as a primary outcome — it is a 1000 ms window and a pure flake source; assert the clipboard content and the modal closing instead, which is what the prior art already does correctly. And never use a fixed `waitForTimeout` to ride out the 100 ms focus delays; a web-first assertion on the resulting state absorbs them.

The fixture corpus

Committed JSON files under `e2e/fixtures/storage/`, each one a whole pre-migration state. These are the raw material for both the "React still reads legacy data" tests and the malformed-input tests.

Fixture	Contents	Purpose
`empty.json`	no keys at all	First-run path; the default template must apply and no key may be created until a template is saved
`single-month.json`	1 group, 4 dates, 1 override	Smallest realistic state; the RP-01 §3 shape verbatim
`multi-month.json`	1 group spanning 3 months with three different override prices	Money arithmetic per month; guards the denormalised `dates` invariant
`many-groups.json`	10 groups over 10 months, 4 lessons each	RP-01 §3 infers this lands near 14 KB; the realistic upper bound of real use
`mixed-currency.json`	2 groups, one `UAH` one `PLN`, settings holding one of them	The global `defaultCurrency` shape question in RP-01 open question 5
`custom-template.json`	synthetic template string with all three tokens	Template is a raw string, not JSON, and is global
`denormalised-drift.json`	`group.dates` and override `dates` deliberately disagreeing	The invariant a typed model must either enforce or eliminate

Every fixture uses only synthetic data. The template fixture must contain a **synthetic** template — never the default template's contents, in whole or in part, because RP-01 D0 records that it holds a real person's name, IBAN and tax identifier at `index.html:387–392` and `400`.

The malformed and adversarial matrix

This is where the layer earns its keep, and where the two projects must be allowed to disagree. Legacy behaviour is a fact; React behaviour is a requirement.

Case	Seeded value	Legacy today, per RP-01	React requirement	Tag
Truncated JSON	`[{"name":"Broken"`	Uncaught `SyntaxError` aborts `init()`; empty group list, no empty state, zero month options, inert buttons (D14)	Renders a recovery surface, never a dead page; original value preserved for export	`DEF`
Not an array	`{"name":"x"}`	Parses, then rendering iterates a non-array	Rejected by a shape guard	`DEF`
Non-3-letter currency	group with `"US Dollar"`	`Intl` throws `RangeError` inside render before the overlay opens, so the group is permanently unopenable (D5)	Group opens; currency is flagged and editable	`DEF`
Malformed month keys	override key `5-08-10`	Persists; the app's own CSV export then fails to re-import with `Invalid month format` (D4)	Detected, surfaced, and exportable without loss	`DEF`
Empty-string template	`paymentTemplate` set to `""`	Falsy, so `getTemplate` silently returns the hardcoded default (D9, masked)	Explicit: either preserved as empty or reset with a visible message	`DEF`
Override with no `dates` property	`{"2026-03":{"price":100}}`	Survives `normalizeOverrides`, because the guard is `data.dates && data.dates.length === 0` (D20)	Same or normalised, but asserted either way	`CHAR`
Override with empty `dates` array	`{"2026-03":{"price":100,"dates":[]}}`	Deleted on save, losing a price set before dates (D20)	Product decision required; pin current behaviour meanwhile	`CHAR`
Duplicate group names	two groups named identically	Distinct in memory by index, silently merged by CSV import (RP-01 §3)	Stable identity that does not depend on array position	`DEF`
Group name containing markup	name with an `img` tag and an error handler	Executes; persists; re-fires on every load (D1)	Escaped by JSX; asserted as inert text	`DEF`
`paymentTemplate` present, other keys absent	template only	Loads with no groups; template survives `clear()` (D15)	Same, and `clear()` behaviour matches its own confirmation text	`CHAR`
Storage unavailable	context with storage blocked	Unguarded; `SecurityError` is a documented outcome when a policy prevents persistence [S30]	Degrades to session-only with a visible warning	`DEF`

Round-trip direction

Test both directions, because rollback is a real scenario for a solo maintainer shipping to one non-technical user.

1. **Forward read.** Seed a legacy fixture, load the React project, assert every visible invariant: card count, `{n} planned lessons`, month row heading, `Total:` and `Per lesson:` strings, and the generated payment message.
2. **Forward write.** From that same seed, perform one mutation in React, read the three keys back, and assert the value is structurally equal to what the legacy app would have written. Compare parsed structures, not strings, so key ordering is not asserted by accident.
3. **Backward read.** Take the state React just wrote, seed it into the legacy project, and assert the same visible invariants. If this fails, a rollback destroys data.
4. **CSV bridge.** Export from legacy, import into React, and the reverse. RP-01 D16 and D17 mean this is already lossy — the export omits the template and has no BOM — so assert the known loss explicitly rather than letting it pass unnoticed.

5. Test data and isolation strategy

Concern	Rule	Rationale
Isolation between tests	One context per test, seeded through the `storageState` option override	"Each test should be completely isolated from another test and should run independently with its own local storage, session storage, data, cookies etc." [S2]
Reset between tests	None needed, and none written	A fresh context starts with empty storage; explicit teardown is redundant work that can itself fail
Data used in assertions	Explicit literals, never generated	A generated value that appears in an expected string makes a failure unreadable
Data not used in assertions	`faker`, seeded from the test title alone	`AGENTS.md` prescribes `faker`; determinism requires the seed not depend on worker count
Group names	Curated pool, not `faker.company.name()`	Company names contain commas, apostrophes and quotes, which collide non-deterministically with the CSV round-trip and with D12's balanced-quote defect
Adversarial names	Explicit cases: one Cyrillic, one containing a comma, one containing a double quote, one containing markup	Each maps to a named RP-01 defect (D17, CSV quoting, D12, D1). Randomness would hit these by accident and flake
Month and date arithmetic	Derived from `FIXED_NOW`, never from `Date.now()`	See below
Personal data	Never in a fixture, a factory default, a snapshot or a report	RP-01 D0. The default template holds a real IBAN and tax identifier

Concern	Rule	Rationale
The end user's real data	Tests never target the GitHub Pages origin	Same-origin `localStorage` writes on the production origin would destroy her only copy

The clock-dependence trap in the current factories

`e2e/ui/support/test-data.ts:31-40` defines `pickMonthContext()` as `faker.date.soon({ days: 240 })`. Seeding `faker` makes the *offset* deterministic but the *base* is wall-clock `now`, so the month a test operates on drifts every day the suite is run. Three consequences, all inference from reading the code together with `index.html:1055-1078`:

- `render.monthlyOverrides` renders a row for every month with lessons **plus** `currentCalendarMonth`, derived from `App.state.calYear` and `calMonth`. Several assertions depend on a row existing for a month with zero lessons — for instance `schedule-editing.spec.ts:208` expects `(0 lessons)` after a cancel. That row exists only because the calendar state still points at the picked month. Change the picked month relative to today and the row set changes.
- A picked month that happens to coincide with the real current month exercises a different code path from one that does not, so a test can pass for months and then fail on a date boundary.
- `payment-messages.spec.ts:112-113` computes `emptyMonthKey` from a live `new Date()` at module scope. That is the only place the suite acknowledges the current month, and it does so by reading the machine clock at collection time.

The fix is one line of policy: derive every month and date from `FIXED_NOW`, and set that same instant with `page.clock.setFixedTime` before navigating. Choose an instant deliberately — mid-month, mid-year, on a weekday, in a 31-day month — so that off-by-one and month-length bugs are reachable. A second suite run pinned to a 28-day February and to a December-to-January boundary is worth two extra project entries rather than randomisation.

Origin isolation, and a latent bug in the current seeding

`localStorage` is scoped per origin, and MDN is explicit that "`localStorage` data is specific to the protocol of the document" [S30]. Playwright's `storageState` therefore matches on the exact origin string. The current prior art couples two independent places to the same literal: `playwright.config.ts` sets `baseURL: 'http://localhost:4173'`, and `e2e/ui/support/storage-state.ts:41` uses that same string as the storageState `origin`. RP-01's own runtime verification used `http://127.0.0.1:4173`.

`http://localhost:4173` and `http://127.0.0.1:4173` are **different origins**. If either side is changed independently, the seeded data lands on an origin the page never reads, every test sees an empty application, and nothing errors — the failures surface as unrelated assertion mismatches. Deriving the origin from `testInfo.project.use.baseURL`, as §4 does, removes the coupling. Standardise on `127.0.0.1` for both, since RP-01 confirmed `window.isSecureContext === true` there, which the clipboard write requires.

On protecting the end user's data: the whole test surface is loopback, and Playwright uses a throwaway browser profile, so the only realistic exposure is a `baseURL` pointed at the production origin. One sentence of policy covers it — tests target loopback only — and it belongs in the config comment, not in a

runtime guard.

6. CI pipeline design

The repository is public, and GitHub states "GitHub Actions usage is **free** for **public repositories** that use standard GitHub-hosted runners" [S29], so runtime cost is not a constraint. Wall-clock feedback time is.

Aspect	Design
Triggers	`push` to `main`, all `pull_request`, and `workflow_dispatch` for manual reruns
Runner	`ubuntu-latest`, single job, `timeout-minutes: 20`
Node	`actions/setup-node@v7` with `node-version: lts/*` and `cache: npm` [S27]. `@playwright/test` 1.62.1 requires `node >= 20` [S6]
Dependency install	`npm ci` against a committed `package-lock.json`
Browser install	`npx playwright install --with-deps chromium` — one engine, plus OS dependencies in one step [S19]
Browser caching	**None.** "Caching browser binaries is not recommended, since the amount of time it takes to restore the cache is comparable to the time it takes to download the binaries" [S17]
Serving the build	Playwright's own `webServer` array starts the static server or servers and waits on their URLs [S14][S15]. No separate CI step
Static checks	`npm run typecheck` before the browser install, so a type error fails in seconds rather than after a download
Test order	unit and component first (fast, no browser), then E2E
Artifacts on failure	`playwright-report/` and `test-results/` via `actions/upload-artifact@v7`, `retention-days: 7`
Machine-readable result	A `json` reporter file inside `test-results/`, which is what §8's exit criterion diffs
Parallelism	`workers: 2` in CI. See the note below
Retries	`retries: 2` in CI, `0` locally, as the prior art already has it

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
```

```
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: lts/*
          cache: npm
      - run: npm ci
      - run: npm run typecheck
      - run: npx playwright install --with-deps chromium
      - run: npm run test:e2e
      - uses: actions/upload-artifact@v7
        if: ${{ !cancelled() }}
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 7
      - uses: actions/upload-artifact@v7
        if: failure()
        with:
          name: test-results
          path: test-results/
          retention-days: 7
```

From Phase 1 onward insert `npm run test:unit` between `typecheck` and the browser install. That script does not exist yet, in either the committed tree or the uncommitted `package.json`, and must be added with Vitest.

Verified action versions, and a documentation disagreement

Action	Version used here	Evidence
`actions/checkout`	`v7`	Every example in the README on `main` uses `actions/checkout@v7`, accessed 2026-08-20 [S26]
`actions/setup-node`	`v7`	Usage section examples all read `actions/setup-node@v7`, accessed 2026-08-20 [S27]
`actions/upload-artifact`	`v7`	README on `main` shows `- uses: actions/upload-artifact@v7`, accessed 2026-08-20 [S28]

Playwright's own CI-setup page still publishes a workflow pinned to `actions/checkout@v6`, `actions/setup-node@v6` and `actions/upload-artifact@v4` [S18]. Both are primary sources and they disagree. Each action's own repository is authoritative for its own current major, so the table above wins; the discrepancy is recorded because a reader comparing this report against Playwright's docs will notice it. Default artifact retention is 90 days [S28]; 7 is chosen deliberately, since these artifacts are only useful while a failure is being diagnosed.

On worker count

`workers: 2` is **my design call**, not an appeal to documentation. The justification is specific to this app: there is no backend, no shared database, no network, and every test gets its own browser context with its own `localStorage`, so the only shared resource is CPU on a two-core runner. Two workers roughly halves wall-clock time with no shared-state risk. If flake appears, reduce to 1 before touching anything else, and record the change. Playwright's CI page does discuss worker counts, but I could not obtain a verbatim sentence to quote, so I decline to cite it as authority for a number.

The gate that does not exist yet

RP-01 established that no workflow file exists and that branch-based GitHub Pages publishing needs none [S33]. So the site deploys directly from the branch, and **CI cannot block a bad deploy** — a green or red check is advisory only. Making tests a real gate requires moving publishing to an Actions workflow so that deployment depends on the test job. The specific action versions for that path are `TBD`; resolve by reading the `actions/configure-pages` and `actions/deploy-pages` READMEs on `main`. This is a genuine architectural gap, not a configuration detail, and it belongs to whichever report owns the deployment pipeline.

Runtime budget

These are **design targets and caps**, not measurements. The suite was not executed in this batch.

Stage	Budget	Enforcement
`typecheck`	under 15 s	none needed
Unit, Vitest node	under 5 s	fails the budget by being noticeable
Component, Vitest jsdom	under 15 s	as above
E2E, one project, chromium, 2 workers	under 4 min	`globalTimeout: 15 * 60 * 1000`
E2E, both projects	under 8 min	same cap
Whole job including install	under 20 min	`timeout-minutes: 20`

Actual current runtime is `TBD`. Resolve by running `npm run test:e2e` in the main checkout and reading the reporter duration — with the caveat from §9 that the existing suite cannot currently pass against the committed `index.html` at all, so any figure obtained today describes the uncommitted working copy.

7. Test plan

Layers: `E2E`, `SC` storage contract, `U` unit, `C` component, `A11Y` axe ratchet, `ARIA` structural snapshot. Tags per §3: `INV`, `DEF`, `CHAR`. Phases: `P0` baseline freeze on the legacy app, `P1` extraction plus unit tests, `P2` dual-project run against React, `P3` post-cutover.

Journey	Layer	Priority	Phase	Tag	Assertion anchors
First load with no data shows the empty state	E2E, SC	P0	P0	INV	Exact string `No groups yet. Click '+ Add Group' to get started!`; no key created
Add a group, card appears with zero lessons	E2E, SC	P0	P0	INV	Card test id; `0 planned lessons`; `groupLessonPlannerData` parsed shape
Add-group defaults are actually the defaults	E2E	P1	P0	CHAR	Fallback name `Untitled Group`; price 0; current `defaultCurrency`
Open a group card, group modal shows info and month rows	E2E, ARIA	P0	P0	INV	Modal container test id; heading `Edit Group`; month row test ids
Edit name, price and currency, save	E2E, SC	P0	P0	INV	Display test ids; card text; both storage keys re-read
Cancel info edit does not revert the price	E2E, SC	P0	P0	DEF	Price display and `group.price` in storage after Cancel (D2)
Committing a price does not discard an unsaved name	E2E	P0	P0	DEF	Name input value after the price `change` fires (D7)
Delete a group after confirming	E2E, SC	P0	P0	INV	`dialog` message contains the name; empty state; key holds `[]`
Delete a group, dismiss the confirm	E2E, SC	P1	P0	INV	Card still present; storage unchanged
Numeric-aware card ordering	E2E	P2	P1	INV	Rendered card order for names with embedded numbers
Open the schedule editor	E2E, ARIA	P0	P0	INV	Calendar container visible; month list hidden
Toggle a single day on and off	E2E	P0	P0	INV	Day-cell test id selected state; summary text
Weekday bulk select, then bulk clear	E2E	P0	P0	INV	Selected count equals weekday occurrences; second click clears
Bulk price writes to every touched month	E2E, SC	P0	P1	CHAR	Per-month override prices after save, including invisible months (D3)
Inline per-month price during editing	E2E, SC	P1	P1	INV	Month price input test id; recomputed `Total:`
Commit the schedule with Done	E2E, SC	P0	P0	INV	Month row counts and totals; both `dates` arrays in storage
Card lesson count refreshes after Done	E2E	P0	P0	DEF	Card count without a reload (D6)

Journey	Layer	Priority	Phase	Tag	Assertion anchors
Discard the schedule with Cancel	E2E, SC	P0	P0	INV	No override created; storage byte-unchanged
Escape discards pending schedule edits silently	E2E	P1	P1	CHAR	Dates and price after Escape; zero dialogs (D13)
Month and year navigation, and Today	E2E	P1	P1	INV	Rendered grid month; `Today` returns to `FIXED_NOW` month
One-digit year cannot corrupt date keys	E2E, SC, U	P0	P0	DEF	Every persisted key matches four-digit-year form (D4)
Clear Month drops only the visible month	E2E, SC	P1	P1	INV	Selected count zero; other months untouched
Month-row deep link opens that month	E2E	P1	P1	INV	Calendar month equals the clicked row's key
Monthly totals arithmetic	U, E2E	P0	P1	INV	`Total:` and `Per lesson:` strings against computed `Intl` output
Generate a payment message from the default template	E2E	P0	P0	INV	Review textarea value; all three tokens substituted
Generate a payment message from a custom template	E2E, U	P0	P0	INV	Synthetic template text present; tokens substituted
Edit the message template, saved globally	E2E, SC	P0	P0	INV	`paymentTemplate` raw string; applies to a second group
Copy the message to the clipboard	E2E	P0	P0	INV	Stubbed clipboard content equals textarea value; modal closes
Clipboard rejection is surfaced	E2E	P1	P2	DEF	Stub rejects; success indicator must not lie (D18)
Export CSV content, not just the filename	E2E, U	P0	P0	INV	Header row exact text; row values; CRLF endings
Export includes a UTF-8 BOM	E2E	P1	P1	DEF	First three bytes of the download (D17)
Export includes the template	E2E	P1	P2	DEF	Template recoverable from the export (D16)
Import a valid CSV replaces all data	E2E, SC	P0	P0	CHAR	Previous groups gone; no confirmation dialog (D11)
CSV round trip preserves everything	E2E, SC	P0	P0	INV	Structural equality of the three keys before and after
CSV parser rejects empty, headerless and bad-month files	U, E2E	P0	P1	INV	Exact alert strings from RP-01 §7; prior data intact

Journey	Layer	Priority	Phase	Tag	Assertion anchors
A balanced stray quote must not destroy data	U, E2E	P0	P1	DEF	Existing groups survive; an error is shown (D12)
Duplicate CSV rows merge and dates dedupe	U	P1	P1	CHAR	Merged group; sorted unique `dates`
`YYYY/M` month normalisation	U	P2	P1	INV	Normalised to `YYYY-MM`
A non-3-letter currency must not brick the group	E2E, SC	P0	P0	DEF	Group opens; no uncaught error (D5)
Clear All Data leaves the template behind	E2E, SC	P1	P0	CHAR	Remaining key set is exactly the template key (D15)
Reload restores groups, overrides, settings and template	E2E, SC	P0	P0	INV	All visible invariants after a reload
Corrupt stored JSON must not produce a dead page	SC, E2E	P0	P0	DEF	Recovery surface rendered; buttons live (D14)
Group name containing markup stays inert	E2E, SC, C	P0	P0	DEF	Rendered as text; no element created; no script side effect (D1)
Pluralisation of lesson and day counts	E2E	P1	P0	DEF	`(1 lesson)`, `1 day selected` (D19)
Closed overlays are outside the tab order	E2E, A11Y	P0	P0	DEF	Tab sweep reaches only toolbar buttons (D8, D9)
Cards and day cells are keyboard operable	E2E, A11Y	P0	P2	DEF	Focusable, activatable by Enter and Space (D10)
Unload prompt fires whenever a group exists	E2E	P2	P1	CHAR	`beforeunload` dialog observed (D24)
Per-view accessibility violation budget	A11Y	P1	P1	DEF	Violation set is a subset of the committed baseline
Per-view accessibility-tree shape	ARIA	P2	P0	INV	Five snapshots, one per view, scoped to the open container

Phase boundaries

Phase 0, baseline freeze. Written entirely against the legacy app. Deliverables: the test-id contract committed in one change; the seeding and clock helpers; every `P0` row above; the five aria snapshots as the before side of the migration diff. Exit condition: the suite runs green against the **committed** `index.html`, with `DEF` rows recorded as expected failures. Nothing about React starts before this holds, because a baseline that cannot run against the deployed artifact is not a baseline — which is precisely how the current scaffold went wrong.

Phase 1, extraction. Move the pure logic out of the inline script into importable modules — RP-01 §5 identifies `services` and `utils` as the cheapest early win — and add the `U` rows. Each extraction is verified by the Phase 0 E2E suite staying green, which is the whole point of ordering it this way.

Phase 2, dual run. Add the `react` project and the second `webServer` entry. The same specs execute twice. The `DEF` rows flip to passing on the React side, one at a time, and each flip is a visible, dated piece of progress.

Phase 3, post-cutover. Delete the `legacy` project and its server entry. Regenerate the aria snapshots as the new baseline. Tighten the a11y budget to the level the React app actually achieves. Retag the `CHAR` rows once the RP-01 open questions are answered.

8. Coverage targets and migration verification exit criteria

Why not a line-coverage percentage

Three specific reasons, not a general preference.

1. **It is not measurable today at any honest cost.** The logic is an inline `<script>` in an HTML file with no build step (RP-01 §5). Getting line coverage means either instrumenting the file or extracting everything first — and if everything is extracted, the migration is largely done.
2. **It would point the tests at code that should be deleted.** RP-01 identifies `App.utils.formatDate` as never called, `groupInfoForm.onsubmit` as unreachable, and the `saveGroup` auto-save-schedule branch as unreachable, plus roughly 35 lines of dead CSS. A coverage number rewards writing tests for those. The prior art already demonstrates the failure mode: LP-010 is a `P0` scenario written against the unreachable branch.
3. **It measures the wrong risk.** The risk is data loss across a rewrite, not untested statements. A suite at 90% lines that never asserts the persisted shape would not catch the thing that matters.

Coverage targets, expressed as workflows

Target	Definition	Measured by
Core journey coverage	100% of the features RP-01 §2 classes as `core` — 23 rows, counted — have at least one E2E test that asserts both a visible outcome and a storage outcome	A checklist keyed to RP-01 §2 rows
Storage key coverage	100% of the three keys in RP-01 §4 asserted for read, write and round trip, in both directions	The `SC` specs
Malformed-input coverage	100% of the eleven rows in §4's adversarial matrix have a test, with an explicit per-project expectation	The `SC` specs
Defect pinning	100% of the RP-01 §8 defects are either fixed with a test proving it, or pinned by a `CHAR` or `DEF` test proving the current behaviour	Cross-reference table from defect id to test title

Target	Definition	Measured by
Pure-function coverage	Every extracted module has unit tests for its documented error strings, from RP-01 §7's alert list	Vitest, per module
Accessibility budget	The violation set never grows	The committed baseline file's diff
Dead code	Zero tests reference `App.utils.formatDate`, `groupInfoForm.onsubmit` or the `saveGroup` calendar branch	A grep in review

The exit criterion, as a command rather than a claim

> **The migration is verified when the same spec set, run against the `legacy` and `react` projects, produces an identical pass-or-fail result for every test, except for the enumerated `DEF` rows, which must fail on `legacy` and pass on `react`.**

That is a diff of two machine-readable reports, which is why §6 adds a `json` reporter. The check reduces to: for each test id, compare the two outcomes against the expected pair from its tag. Any test that passes on `legacy` and fails on `react` is a regression and blocks the cutover. Any `DEF` row that fails on `react` means the migration has not yet delivered its stated benefit. Any test that passes on both but was tagged `DEF` means the tag or the legacy behaviour was misunderstood, which is itself a finding worth surfacing.

Four preconditions make that boolean trustworthy, and none of them is a percentage:

1. The suite runs green against the **committed** `index.html`, not a working copy.
2. Every test derives its dates from the fixed clock; no test reads `Date.now()`.
3. No retained test locates an element by `#id`, class, XPath, or `page.evaluate` reach-in.
4. The three storage keys and the persisted shape are asserted by name and by parsed structure, in both directions.

9. Assessment of the existing uncommitted test scaffold

All of this is **uncommitted local work** in the main checkout, cited as evidence of the developer's intent and never as authority. It is, on the whole, better than the repository's committed state suggests: the seeding strategy, the dialog handling, the clipboard stub and the `timezoneId` pin are all correct instincts that this report keeps.

The finding that matters most

The suite cannot pass against the deployed application. `e2e/ui/pages/planner-page.ts:25` locates a group card as `[data-group-name=…]`; `e2e/ui/pages/monthly-overrides.ts:13` locates a month row as `[data-month-key=…]`; `e2e/ui/pages/calendar-editor.ts:32,36` locate day cells and weekday headers by `data-date` and `data-weekday`; and five methods read text through `getByTestId`. RP-01 runtime-verified that the committed and deployed `index.html` has **zero** `data-testid` and zero `data-*`

attributes, and my own diff of the two files confirms every one of those attributes is added only by the uncommitted working copy. Opening a group card is a precondition of most specs, so the majority of the suite fails at its first action against the file users load.

The consequence for the record: the claim in `docs/qa-coverage-investigation.md` that `npm run test:e2e` passed 22/22 on 20 March 2026 can only describe the 1491-line working copy. It is not evidence about the deployed app, and it should not be cited as such. This is the same class of error as the document's line-number drift that RP-01 identified.

LP-010, verified false independently

RP-01 reported this from a runtime session; I re-derived it statically from the committed file, and the mechanism is airtight.

`index.html:673–675` contains the branch under test — `saveGroup` calls `saveDateChanges()` when `calendarContainer.style.display === 'block'`. Reaching it requires clicking `#saveGroupBtn` while the calendar is open. But `startEditingDates` (`index.html:720–744`) sets `App.state.isEditing = true` and `App.state.isEditingGroupInfo = false`, then calls `App.render.groupInfo()` at line 742. Inside `render.groupInfo`, `isEditingInfo` is the logical OR of `App.state.isEditingGroupInfo` and `isAdding`, which is now `false`, so line 1015 applies `.hidden` to `#groupInfoForm` — the form containing `#saveGroupBtn`. Line 1016 then applies `.hidden` to `#editGroupInfoBtn` because the expression includes `App.state.isEditing`. So while the calendar is open the Save button is inside a `display: none` container **and** the only control that could re-reveal it is itself hidden. The branch is unreachable through the user interface, exactly as RP-01 found.

What that implies about the rest of that document. The error is diagnostic of method, not of care: LP-010 was derived by reading the code rather than exercising it, and the line numbers throughout resolve against a file that was never deployed. So every *behavioural* claim in `docs/qa-coverage-investigation.md` must be re-verified before it is relied on — including the "22/22" run and the "throws on missing columns, malformed CSV, or invalid month values" claim that RP-01 already found partly false. Its *test-design* judgements are a different matter and hold up independently of the app: the observation that the ARIA snapshots are low-signal on the primary assertion path, that "Export CSV when groups exist" only checks a filename, that CSV parsing and message rendering belong at a lower layer once extracted, and its list of missing high-value scenarios are all correct and this report adopts them. Use the document as a well-reasoned coverage analysis; do not use it as a description of the deployed app.

Keep, fix, drop

Item	Verdict	Reasoning
Seeding `localStorage` before load via `storageState`	**Keep** — this is the right mechanism	The app reads storage inside `init()` before rendering; seeding after navigation would be too late
`faker` for generated data	**Keep**, per `AGENTS.md`	With the seeding and name-pool corrections in §5

Item	Verdict	Reasoning
Atomic tests, no shared state	**Keep**, per `AGENTS.md` and [S2]	Correctly implemented; each test gets its own context
`timezoneId: 'UTC'`	**Keep**, and add `locale: 'en-US'`	Removes the runner's timezone; locale removes the runner's number formatting
Dialog handling via `waitForEvent('dialog')` before the action	**Keep**	Correct ordering, and it asserts the message text, which RP-01 §7 lists as a stable anchor
Clipboard stub via `context.addInitScript`	**Keep**	Runs "after the document was created but before any of its scripts were run" [S20], which is the right hook
`webServer` config and the port-4173 convention	**Keep**	Matches `AGENTS.md`; extend to an array in Phase 2
The QA document's coverage-gap analysis	**Keep** as analysis	Independently sound; see above
Page objects anchored on `#id` CSS selectors	**Fix — highest priority**	38 `page.locator('#…')` calls across the six page objects, counted by grep. Per [S1] these are the least resilient strategy available, and none survives a React rewrite
Locators depending on `data-*` from the uncommitted file	**Fix**	Blocks the suite from running against the deployed app at all. Resolve by committing the test-id contract, per §3 Step B
Custom `context` fixture built with `browser.newContext()`	**Fix**	Bypasses Playwright's own context fixture, so `use` options are applied only partially and video is not recorded despite `video: 'retain-on-failure'` in the config [S31]. Override the `storageState` **option** instead, which is the documented route [S12][S13]
`resolvedBaseURL` reading `testInfo.project.use` with a hardcoded `'http://localhost:4173'` fallback	**Fix**	A silent fallback to a possibly-wrong origin. Derive the origin from the project's `baseURL`, per §4
`localhost` in the config versus `127.0.0.1` used in RP-01's verification	**Fix**	Different origins for `localStorage` [S30]. A mismatch produces an empty app with no error
`faker.seed(seedFromTitle(title) + workerIndex)` in `beforeEach`	**Fix**	Makes generated data depend on the number of workers, so a shard-count change alters test data

Item	Verdict	Reasoning
Module-scope `faker.seed(...)` plus data computed at collection time	**Fix**	Fixture data is built when the file is loaded, not when the test runs, which interacts badly with the per-test reseed above
`pickMonthContext()` as `faker.date.soon({days: 240})`	**Fix — a dated time bomb**	Wall-clock dependent; see §5
No clock control anywhere	**Fix**	The app reads `new Date()` at module init and in five handlers (RP-01 §6). Add `page.clock.setFixedTime` before `goto` [S7][S8]
Specs asserting `1 days selected` and `(1 lessons)`	**Fix**	RP-01 quick win 5 will turn these red. Retag as `DEF` and assert the corrected strings
`e2e/features/csv-import-export.spec.ts` export test asserting only the filename	**Fix**	CSV is the only backup path and RP-01 D4, D12, D16 and D17 all live in it. Assert content and round trip
`workers: 2` in CI	**Keep, with a documented fallback**	Justified in §6; drop to 1 first if flake appears
No `projects` in the config	**Fix**	Currently chromium-only implicitly. Make it explicit, then use projects for the legacy-versus-React matrix
The screenplay layer: actor, abilities, tasks, questions, assertions	**Drop, or collapse**	412 lines across 17 files wrapping six page objects for seven spec files, and `AGENTS.md` prescribes page objects. Each user action is currently expressed three times. For a solo maintainer this is indirection without payoff — the pattern earns its keep on large teams with a shared vocabulary, not here. Collapse to page objects plus plain async helpers
`e2e/visual.spec.ts-snapshots/`	**Drop**	An empty directory with no corresponding spec, and `testDir: 'e2e/features'` would exclude a spec at that path anyway. Dead scaffolding
Five ARIA snapshots on the primary assertion path	**Reduce to five total, off the primary path**	The developer's own QA document reaches the same conclusion. Two of them embed emoji glyphs and a full twelve-option month list, which churn on unrelated changes
`tsconfig.json` with `module: commonjs`	**Note only**	Works, and typechecks. ESM is the norm for Vitest and the React toolchain; revisit at Phase 1, not before

10. Risks, gaps and unknowns

#	Risk or unknown	Impact	Resolution
R1	**Blocks later reports.** The test-id contract requires editing `index.html`, but RP-01 open question 1 records that it is unknown whether the uncommitted working copy is meant to land	Blocks Phase 0 entirely. Strategy A cannot start without anchors, and the existing suite cannot go green without them	Ask the developer. This is the single highest-leverage question in the programme
R1b	**Blocks later reports.** `playwright.config.ts` declares `baseURL: 'http://localhost:4173'` while `e2e/ui/support/storage-state.ts:41` reuses that same literal as the storageState origin, and RP-01's runtime verification used `http://127.0.0.1:4173`	Different origins for `localStorage` [S30]. Any independent change to either side seeds an origin the page never reads, producing an empty app with **no error** and misleading assertion failures	Standardise on `127.0.0.1` and derive the origin from the project's `baseURL`, per §4 and quick win 2. Any later report that specifies a serving origin must use the same one
R2	RP-01 open question 3: the cross-month bulk price bleed (D3) has no agreed intended behaviour	A `CHAR` test now, but the pricing tests cannot become invariants until it is settled	Product decision
R3	The end user's browser and platform are unknown	Determines whether the WebKit smoke lane is necessary or theatre	Ask the teacher. `TBD`
R4	Real dataset scale is unknown; RP-01 records the original data as destroyed	The `many-groups.json` fixture size is an inference, not a measurement	Ask the teacher. `TBD`
R5	`@axe-core/playwright` current version not verified	The a11y layer's dependency pin is unspecified	Read `https://registry.npmjs.org/@axe-core/playwright/latest`. `TBD`
R6	`actions/configure-pages` and `actions/deploy-pages` versions not verified	Blocks turning CI into a real deploy gate	Read each action's README on `main`. `TBD`
R7	Actual suite runtime unmeasured	The §6 budget is a target, not a projection	Run `npm run test:e2e`, noting it currently exercises the uncommitted file. `TBD`
R8	`localStorage` quota for this origin is unmeasured (carried from RP-01)	Determines whether a quota-exceeded contract test is worth writing	`navigator.storage.estimate()` in the target browser. `TBD`

#	Risk or unknown	Impact	Resolution
R9	Whether `baseURL` can be destructured directly as a fixture dependency is not stated on Playwright's fixtures page [S4]	Affects one line of the seeding fixture	§4 sidesteps it by reading `testInfo.project.use.baseURL`. Confirm with `npm run typecheck`
R10	Automatic-fixture ordering relative to the lazily created `page` fixture is not documented	A clock installed too late fails silently rather than loudly	§4 sidesteps it with an explicit `openApp` helper. Do not use an auto fixture here
R11	Playwright component testing changed shape in 1.62; the story-gallery contract is new	A future reader may find guidance written against the older experimental packages	Version-sensitive. Re-read [S3] and [S4] before revisiting the §2 decision
R12	`@playwright/test` is pinned as `^1.51.0` in the uncommitted `package.json`, so a fresh install resolves to 1.62.x	Eleven minor versions of behaviour change arrive silently on the first `npm install`	Pin exactly, run the suite, then raise deliberately. Note `node >= 20` is now required [S6]
R13	Two projects share one `test-results/` output directory	Fixture files written with `testInfo.outputPath` are already per-test, so this is low risk, but report merging needs care	Verify once the second project exists
R14	The `DEF` tag encodes an expectation about software that does not exist yet	A misjudged `DEF` row looks like a migration failure	Review the `DEF` list with the developer before Phase 2, not during it
R15	Sources disagree on GitHub Action versions: Playwright's CI page publishes v6/v6/v4 [S18] while each action's own README publishes v7 [S26][S27][S28]	A reader may conclude the report is wrong	Each action's repository is authoritative for its own version. Both readings are recorded in §6
R16	CI is advisory only, because branch-based Pages publishing has no build step to gate [S33]	A red suite cannot stop a deploy today	See R6

Contradictions with the inputs to this report

Input claim	Correction	Why it matters
The brief lists `e2e/visual.spec.ts-snapshots/` among the prior art	The directory is **empty and orphaned**. There is no `e2e/visual.spec.ts`, and `testDir: 'e2e/features'` would not collect one at that path anyway	Visual regression was attempted and abandoned, not implemented. It is dead scaffolding, not a layer to assess

Input claim	Correction	Why it matters
The brief says the working copy "adds seven `data-testid` attributes"	**Eight** distinct values, counted by grep over the working copy, plus six `data-*` dataset attributes	The test-id contract in §3 must enumerate all eight or the React components will be one anchor short
The brief describes the working copy as adding "a modal-visibility fix"	Accurate, and the diff confirms it: `hidden` on all three overlays, a `[hidden]` display rule, and `hidden` toggling in six open and close handlers	Confirms RP-01 quick win 2 is already written and exercised, so adopting it carries no design risk
`docs/qa-coverage-investigation.md` records `npm run test:e2e` passing 22/22	Cannot describe the deployed app, because the locators depend on attributes absent from the committed file	Do not cite that run as evidence of any behaviour of the deployed app
`docs/qa-coverage-investigation.md` scenario LP-010, rated P0	False. Re-derived statically here, independently of RP-01's runtime finding	A P0 test written against unreachable code. See §9

11. Sources

#	Title	URL	Accessed	Supports
S1	Playwright — Locators	https://playwright.dev/docs/locators	2026-08-20	Recommended locator priority; CSS and XPath named a bad practice; the getBy* list in §3
S2	Playwright — Best Practices	https://playwright.dev/docs/best-practices	2026-08-20	Test user-visible behaviour; per-test isolation including local storage; web-first assertions
S3	Playwright — Components	https://playwright.dev/docs/test-components	2026-08-20	Component testing replaces the experimental ct packages; no experimental package to depend on; story gallery requirement
S4	Playwright — Fixtures API	https://playwright.dev/docs/api/class-fixtures	2026-08-20	`fixtures.mount()` added in v1.62; gallery page exposing window.mount and window.unmount; built-in fixture list
S5	Playwright — Release notes	https://playwright.dev/docs/release-notes	2026-08-20	1.62 is current; new component-testing model; Clock API in 1.45; aria snapshots in 1.49
S6	npm registry — @playwright/test latest	https://registry.npmjs.org/@playwright/test/latest	2026-08-20	Version 1.62.1 and engines node >= 20, used for the version-pin risk R12

#	Title	URL	Accessed	Supports
S7	Playwright — Clock guide	https://playwright.dev/docs/clock	2026-08-20	Install the clock before navigating; setFixedTime preferred over install unless pausing is needed
S8	Playwright — Clock API	https://playwright.dev/docs/api/class-clock	2026-08-20	setFixedTime keeps timers running; install replaces timer functions; both added in v1.45
S9	Playwright — Visual comparisons	https://playwright.dev/docs/test-snapshots	2026-08-20	Snapshots are named per browser and platform because rendering differs; the basis for rejecting pixel diffs
S10	Playwright — Accessibility testing	https://playwright.dev/docs/accessibility-testing	2026-08-20	@axe-core/playwright is the recommended engine; automated scans catch only some problems
S11	Playwright — TestOptions	https://playwright.dev/docs/api/class-testoptions	2026-08-20	storageState accepts an object with origins and localStorage entries; timezoneId, locale, baseURL semantics
S12	Playwright — Test fixtures	https://playwright.dev/docs/test-fixtures	2026-08-20	Fixtures can be overridden, with storageState as the worked example; option fixtures and automatic fixtures
S13	Playwright — Authentication	https://playwright.dev/docs/auth	2026-08-20	storageState supplied by a fixture defined in test.extend, the pattern §4 adopts
S14	Playwright — TestConfig	https://playwright.dev/docs/api/class-testconfig	2026-08-20	webServer type is Object or Array of Object; globalTimeout and per-test timeout semantics
S15	Playwright — Web server	https://playwright.dev/docs/test-webserver	2026-08-20	Verbatim multi-server array example used for the dual-project config
S16	Playwright — TestProject	https://playwright.dev/docs/api/class-testproject	2026-08-20	Per-project `use` accepts test options, which is what makes per-project baseURL work
S17	Playwright — Continuous Integration	https://playwright.dev/docs/ci	2026-08-20	Caching browser binaries is not recommended because restore time matches download time
S18	Playwright — Setting up CI	https://playwright.dev/docs/ci-intro	2026-08-20	The official GitHub Actions workflow, and the action-version disagreement recorded in §6 and R15

#	Title	URL	Accessed	Supports
S19	Playwright — Browsers	https://playwright.dev/docs/browsers	2026-08-20	`install --with-deps chromium` installs browser and OS dependencies together
S20	Playwright — BrowserContext API	https://playwright.dev/docs/api/class-browsercontext	2026-08-20	context.clock added in v1.45; addInitScript runs after document creation but before page scripts
S21	Testing Library — About Queries	https://testing-library.com/docs/queries/about/	2026-08-20	Query priority with getByRole first and getByTestId as the final resort
S22	npm registry — @testing-library/react latest	https://registry.npmjs.org/@testing-library/react/latest	2026-08-20	Version 16.3.2 and React 18 or 19 peer range for the component layer
S23	npm registry — vitest latest	https://registry.npmjs.org/vitest/latest	2026-08-20	Version 4.1.11 and supported Node versions for the unit layer
S24	Vitest — Browser Mode	https://vitest.dev/guide/browser/	2026-08-20	Provider list, Playwright recommended; no experimental admonition on the page as read
S25	Vitest — Vitest 4 announcement	https://vitest.dev/blog/vitest-4	2026-08-20	Browser Mode's experimental tag removed; toMatchScreenshot added
S26	actions/checkout README on main	https://raw.githubusercontent.com/actions/checkout/main/README.md	2026-08-20	All usage examples pin actions/checkout@v7
S27	actions/setup-node README on main	https://raw.githubusercontent.com/actions/setup-node/main/README.md	2026-08-20	Usage pins actions/setup-node@v7; the `cache` input and lts/* syntax
S28	actions/upload-artifact README on main	https://raw.githubusercontent.com/actions/upload-artifact/main/README.md	2026-08-20	Usage pins actions/upload-artifact@v7; 90-day default retention, 1 to 90 allowed
S29	GitHub Docs — Billing for GitHub Actions	https://docs.github.com/en/billing/concepts/product-billing/github-actions	2026-08-20	Actions usage is free for public repositories on standard GitHub-hosted runners
S30	MDN — Window.localStorage	https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	2026-08-20	Origin and protocol scoping; UTF-16 string values; SecurityError when persistence is blocked

#	Title	URL	Accessed	Supports
S31	microsoft/playwright issue 9002	https://github.com/microsoft/playwright/issues/9002	2026-08-20	Config `use` options apply only partially to a manually created browser.newContext(), with video not recorded
S32	Playwright — Test use options	https://playwright.dev/docs/test-use-options	2026-08-20	Where trace, screenshot and video options are declared, for the §9 artifact-loss finding
S33	GitHub Docs — Configuring a publishing source for GitHub Pages	https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site	2026-08-20	Branch-based publishing requires no in-repo workflow, so CI cannot gate the deploy today

12. Quick wins

Each item below is independently shippable, touches no unresolved decision in this report, and does not require editing `index.html`. Deliberately excluded: committing the `data-testid` set, which is blocked by RP-01 open question 1; collapsing the screenplay layer, which is a refactor rather than a quick win; and adding the React project, which needs a React build to exist.

A necessary caveat for all five prompts. `package.json`, `playwright.config.ts`, `tsconfig.json` and everything under `e2e/` exist as **uncommitted local work in the main checkout** and are not in the committed tree, which contains only `index.html`, `AGENTS.md`, `LICENSE`, `.gitignore` and `docs/`. Each prompt states this where it applies.

Rank	Quick win	Effort	Impact	Basis of ranking
1	Add a GitHub Actions workflow that typechecks and runs the suite	XS	High	Nothing runs automatically today; RP-01 confirms no `.github/` exists. Creates only new files, cannot break the app, and gives every later change a signal
2	Replace the custom `context` fixture with a `storageState` option override	S	High	Restores failure artifacts that the config already asks for but never gets [S31], and removes the hardcoded-origin fallback in the same edit. No spec changes
3	Freeze the clock before navigation and derive test months from it	S	High	Removes the whole class of clock flake at once: module-init `new Date()`, the today highlight, the always-rendered current-month row, and the `faker.date.soon` time bomb

Rank	Quick win	Effort	Impact	Basis of ranking
4	Assert CSV export content, not just the filename	S	High	CSV is the only backup path and four RP-01 defects live in it. The current test would pass while the export was completely wrong
5	Add a storage contract spec for the three keys	S	Medium	Pins the migration's actual contract before any React code exists, and needs no source change because seeding and reading are both test-side

```
PROMPT QW-1: Add a CI workflow that typechecks and runs the Playwright suite
Context: Repo lesson-planner. The committed tree contains only index.html, AGENTS.md,
LICENSE, .gitignore and docs/. There is no .github directory and no CI of any kind, so no
check runs on push or pull request. The Playwright suite, package.json and
playwright.config.ts currently exist as uncommitted local work in the main checkout;
package.json already defines the scripts typecheck and test:e2e. This prompt creates a new
file and assumes the uncommitted test scaffold is committed first.
Task: Create a new file .github/workflows/ci.yml defining a single job on ubuntu-latest with
timeout-minutes 20, triggered on push to main, on all pull_request events and on
workflow_dispatch. Steps in order: actions/checkout@v7; actions/setup-node@v7 with node-
version lts/* and cache npm; npm ci; npm run typecheck; npx playwright install --with-deps
chromium; npm run test:e2e; then actions/upload-artifact@v7 with if always-unless-cancelled
uploading playwright-report/ as playwright-report with retention-days 7; then actions/upload-
artifact@v7 with if failure uploading test-results/ as test-results with retention-days 7.
Constraints: Do not modify index.html. Do not change the three localStorage key names
groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the
persisted data shape. Do not add a browser-binary cache step; Playwright documents that
caching browser binaries is not recommended. Install chromium only, not all browsers. Do not
add a deploy or Pages-publishing step; publishing is branch-based today and changing it is
out of scope. Do not add new npm dependencies. Confirm .gitignore already excludes test-
results/, playwright-report/ and node_modules/ and add any that are missing. Expect the suite
itself to be RED on the first run: the committed index.html has no data-testid or data-*
attributes, so the existing page objects cannot locate group cards or month rows. Do not make
it green by committing the modified index.html, and do not make it green by skipping tests.
The value of this change is that the red is now visible and dated; turning it green is the
separate test-id-contract decision.
Acceptance criteria: The workflow file parses as valid YAML. A pull request triggers exactly
one job. The job runs every step in the declared order and reaches the artifact steps
regardless of whether the test step passed. Both playwright-report and test-results artifacts
appear on the run summary for a failing run, and playwright-report appears for any run that
was not cancelled. The job finishes inside its 20-minute timeout. No step references a
browser other than chromium. No test is skipped, quarantined or marked fixme to obtain a
green result.
Verification: Push a branch and open a pull request; confirm the workflow triggers, the job
runs to completion, and both artifacts are downloadable. Download playwright-report and
confirm the failures listed are locator failures on the missing data attributes, which is the
expected starting state rather than a workflow defect.
```

```
PROMPT QW-2: Seed localStorage by overriding the storageState option instead of building a
context
Context: Repo lesson-planner. The following files are uncommitted local work in the main
checkout, not in the committed tree: e2e/ui/fixtures/test.ts, e2e/ui/support/storage-
state.ts, playwright.config.ts. e2e/ui/fixtures/test.ts currently overrides Playwright's
built-in context fixture and calls browser.newContext with a storageState object. Two
```

```
consequences: config options declared under use are only partially applied to a manually created context, so video is not recorded even though playwright.config.ts sets video retain-on-failure; and the resolvedBaseURL fixture reads testInfo.project.use.baseURL with a hardcoded fallback of http://localhost:4173, which silently seeds the wrong origin if the config baseURL ever changes. Note that http://localhost:4173 and http://127.0.0.1:4173 are different origins for localStorage purposes.
Task: Remove the custom context fixture and the custom page fixture from e2e/ui/fixtures/test.ts, and instead override Playwright's built-in storageState option with a fixture that depends on the existing plannerState option. Inside that fixture, derive the origin by passing testInfo.project.use.baseURL through the URL constructor and reading its origin property, with no string fallback: if baseURL is absent, throw an explicit error naming the missing option. Keep buildStorageState in e2e/ui/support/storage-state.ts as the single place that knows the three key names, and change its signature to accept the derived origin. Keep the clipboard option working by moving stubClipboard onto the built-in context fixture as an override that calls addInitScript and then delegates, or by calling it from an automatic fixture; do not reconstruct the context. Change playwright.config.ts baseURL from http://localhost:4173 to http://127.0.0.1:4173 and keep the webServer port at 4173.
Constraints: Do not modify index.html. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape or the raw-string encoding of paymentTemplate. Do not change any spec file in e2e/features. Do not change the acceptDownloads or timezoneId behaviour that tests already rely on. Do not add new npm dependencies. Never place the contents of the app's default payment template into any fixture.
Acceptance criteria: No file under e2e/ contains a call to browser.newContext. No file under e2e/ contains the literal string http://localhost:4173. npm run typecheck passes. Every spec in e2e/features runs with the same pass or fail result as before the change. On a deliberately failing test, the HTML report contains a trace, a screenshot and a video attachment. Seeding still works, demonstrated by a test that seeds one group and sees it rendered.
Verification: Run npm run typecheck, then npm run test:e2e and compare the pass or fail set against a run recorded before the change. Then make one assertion fail deliberately, run npx playwright show-report, and confirm all three artifact types are attached to that test. Revert the deliberate failure.
```

```
PROMPT QW-3: Freeze the clock before navigation and derive all test dates from it
Context: Repo lesson-planner. The following are uncommitted local work in the main checkout, not in the committed tree: e2e/ui/fixtures/test.ts, e2e/ui/support/test-data.ts, e2e/ui/support/formatters.ts and the seven specs in e2e/features. The application reads new Date() while its inline script initialises, at index.html:369-370, and again in today highlighting, in the Today button, in the schedule editor's default month, in the current-month cutoff of updateDefaultPrice, and in the CSV export filename. The tests currently exercise no clock control at all. Separately, pickMonthContext in e2e/ui/support/test-data.ts uses faker.date.soon with a 240-day window, so although faker is seeded the base instant is the real wall clock and the month under test drifts every day; payment-messages.spec.ts also computes a month key from a live new Date() at module scope.
Task: Add an exported constant FIXED_NOW to e2e/ui/support/test-data.ts holding a single deliberate instant that is mid-month, on a weekday, in a 31-day month. Add an exported helper that takes a Page, calls page.clock.setFixedTime(FIXED_NOW) and then calls page.goto('/'), and use it as the single navigation entry point for every spec. Remove the custom page fixture's implicit navigation if the previous prompt has not already done so. Rewrite pickMonthContext and any other month or date derivation to compute offsets from FIXED_NOW using explicit month arithmetic instead of faker.date, keeping the function signature and return shape unchanged so specs need no edits beyond navigation. Replace the module-scope new Date() in payment-messages.spec.ts with a derivation from FIXED_NOW.
Constraints: Use setFixedTime, not clock.install and not pauseAt: the application depends on real timers elapsing, including a 1000 ms Copied label window and three 100 ms focus delays,
```

```
and freezing them would hang the copy flow. The clock must be set before the first
navigation, because the application reads the date during module initialisation. Do not
modify index.html. Do not change the three localStorage key names or the persisted data
shape. Do not introduce a fake-timer library; page.clock is built in. Do not add
waitForTimeout anywhere. Keep faker for non-asserted values, per AGENTS.md.
Acceptance criteria: No file under e2e/ calls faker.date. No spec file constructs a bare new
Date() to derive a month or year under test. Every spec navigates through the new helper.
Running the suite twice on different calendar days produces identical selected months,
identical month-row headings and identical calendar summary strings. The clipboard test still
passes, proving timers still elapse. npm run typecheck passes.
Verification: Run npm run test:e2e and record the resolved month for one schedule test from
the report. Then set the machine clock forward by 90 days, or run the suite again on a later
date, and confirm the same month appears. Confirm the copy-payment-message test still passes,
which proves the 1000 ms timer still fires.
```

```
PROMPT QW-4: Assert exported CSV content and a full round trip, not just the filename
Context: Repo lesson-planner. e2e/features/csv-import-export.spec.ts is uncommitted local
work in the main checkout, not in the committed tree. Its Export CSV when groups exist test
currently asserts only that the suggested filename matches a lesson-planner prefix pattern,
so the export could produce entirely wrong content and the test would still pass. CSV is the
application's only backup path. The export format is verified as: a header row reading
exactly "Name","Default Price","Currency","Month","Month Price","Dates" with every field
quoted, CRLF row endings, no UTF-8 byte order mark, and ISO dates space-delimited inside one
quoted field.
Task: Extend the export test to read the downloaded file's bytes using the Playwright
download object, and assert the exact header row text, the CRLF row endings, the number of
data rows, and the values of all six columns for a seeded group with two months at different
prices. Add a separate round-trip test that seeds a known group, exports, clears all data
through the UI, imports the exact file that was just downloaded, and then asserts structural
equality of the parsed contents of groupLessonPlannerData and groupLessonPlannerSettings
against their values before the export. Record the current absence of a UTF-8 byte order mark
as an explicit assertion on the first three bytes so that adding one later is a deliberate,
visible change rather than a silent one.
Constraints: Do not modify index.html; this prompt only observes the current export format
and does not change it. Do not change the three localStorage key names
groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the
persisted data shape. Do not assert on the template, which the export is known not to
include. Compare parsed structures rather than JSON strings so that key ordering is not
asserted by accident. Use a seeded group name drawn from an explicit literal, not from faker,
so a quote or comma in a generated name cannot make the assertion non-deterministic. Do not
add new npm dependencies. Never use the contents of the application's default payment
template in any fixture.
Acceptance criteria: The export test fails if the header row text changes by a single
character. The export test fails if row endings change from CRLF to LF. The round-trip test
fails if any group name, default price, currency, month key, month price or date is lost or
altered. The byte-order-mark assertion states the current state explicitly and names the
defect it tracks in a comment. npm run typecheck passes and the whole suite still runs.
Verification: Run npm run test:e2e. To prove the header assertion is actually sensitive,
change one character of the EXPECTED header string inside the test, confirm the test fails on
that exact assertion, and revert the test. Do not edit index.html to test sensitivity. Then
run the round-trip test and confirm it passes.
```

```
PROMPT QW-5: Add a storage contract spec for the three localStorage keys
Context: Repo lesson-planner. The e2e directory is uncommitted local work in the main
checkout, not in the committed tree; this prompt adds a new spec file to it. The application
```

```
persists everything in exactly three localStorage keys: groupLessonPlannerData holding
JSON.stringify of an array of groups, groupLessonPlannerSettings holding JSON.stringify of an
object with a defaultCurrency property, and paymentTemplate holding a raw string that is not
JSON. There is no schema version field, no validation on load and no error handling. Each
group has name, price, currency, dates and monthlyOverrides, where monthlyOverrides is keyed
YYYY-MM and each value has price and dates, and every date string is stored twice, once in
group.dates and once in the relevant override's dates array. No test currently asserts any of
this directly.
Task: Create a new spec file e2e/features/storage-contract.spec.ts and a new directory
e2e/fixtures/storage containing at least three committed JSON fixture files: an empty state,
a single group with four dates in one month, and a group spanning three months with three
different override prices. For each fixture, write a test that seeds it, loads the
application, and asserts the visible invariants that depend on it: the group card count, the
{n} planned lessons text, each month row heading with its lesson count, and each month's
Total and Per lesson strings. Then write a write-side test that performs one schedule change
through the UI and reads all three keys back, asserting the parsed structure including that
group.dates and the union of all override dates arrays contain the same set of date strings.
Add one test that seeds paymentTemplate with a synthetic template containing all three tokens
and asserts it is used in the generated message for two different groups, proving the
template is global.
Constraints: This spec must run against the COMMITTED index.html, which has no data-testid
and no data-* attributes, so it must not use the existing page objects and must not use data-
testid, data-* attributes, id selectors, class selectors or XPath. Locate only by visible
text and read state only by page.evaluate against the three key names by literal string. That
keeps the spec independent of the separate test-id-contract decision. Do not modify
index.html. Do not change the three localStorage key names and do not change the persisted
data shape; this spec describes the existing contract and must not require a source change to
pass. Remember that paymentTemplate is a raw string and must not be JSON-encoded when seeded.
Never place any part of the application's default payment template into a fixture or an
assertion; use synthetic template text only, because the default template contains a real
person's payment identifiers. Use explicit literal group names, not faker, for anything
asserted. Compare parsed structures, not JSON strings. Do not add new npm dependencies.
Acceptance criteria: Three or more fixture JSON files are committed under
e2e/fixtures/storage. Every fixture has at least one test that asserts its visible invariants
using visible text only. The write-side test fails if either dates array diverges from the
other. The template test proves the template applies across two groups. The spec contains
zero occurrences of getByTestId, of the substring data-, of a selector beginning with a hash
character, and of a selector beginning with a dot. No fixture file or assertion contains an
IBAN-shaped string, a digit run of eight or more characters, or any Cyrillic text other than
deliberately synthetic content. npm run typecheck passes and the new spec runs green against
the committed index.html.
Verification: Run npx playwright test e2e/features/storage-contract.spec.ts against a locally
served copy of the committed index.html and confirm the new spec passes with no other spec
involved. Then hand-edit one fixture so a single override date is missing from group.dates,
confirm the write-side invariant assertion fails, and restore the fixture. Finally grep the
new fixture files and spec for the patterns [A-Z]{2}[0-9]{6,} and [0-9]{8,} and confirm zero
matches.
```